

Mutual Fund Analytics Platform

Final Capstone Report

Bluestock Fintech Internship — Capstone Project I

By Avika Aswal

Project Manager: Yash Kale

August 2026

github.com/Avika26/mutual-fund-capstone

Table of Contents

1. Introduction
 2. Data Sources
 3. How I Built the ETL Pipeline
 4. Tools and Architecture
 5. What I Found in EDA (Day 3)
 6. Performance Analysis (Day 4)
 7. Advanced Analytics and Risk Metrics (Day 5)
 8. The Power BI Dashboard
 9. How I Approached Debugging and Verification
 10. Limitations
 11. Recommendations
 12. Conclusion
- Appendix A: Repository Structure
- Appendix B: Deliverables Checklist

1. Introduction

This report covers the mutual fund analytics platform I built for my Capstone Project I at Bluestock Fintech. Over about a week, I worked through the whole pipeline: pulling in data from AMFI and mfapi.in, cleaning it and loading it into a SQLite database, doing exploratory analysis, computing performance metrics, building out some advanced risk analytics, and finally putting it all together in a four-page Power BI dashboard.

The dataset covers 40 mutual fund schemes, with daily NAV data running from January 2022 to May 2026. I calculated daily returns, CAGR, Sharpe and Sortino ratios, Alpha and Beta against Nifty 100, maximum drawdown, a composite fund scorecard, Value-at-Risk, investor cohort behavior, SIP continuity, and built a simple fund recommender.

One thing that came up again and again while I was working through this is that the NAV and benchmark data in this dataset don't behave like real market data. I found near-zero correlation between funds that track the same index, Beta values against Nifty 100 that are basically zero, and R^2 values around 0.0006 — meaning the regression barely explains anything. My best guess is that the NAV series were generated independently of each other rather than being derived from real, shared market movements. I don't think this is a flaw in my analysis — I checked my formulas carefully and cross-verified wherever I could — but it does mean I'd treat this project as a demonstration that I can build a correct analytics pipeline, not as a source of real investment advice. I go into this more in Section 10.

Along the way I ran into a few genuine bugs, not just data quirks — a units mislabeling in the AUM data, a relationship in my Power BI model that was silently inflating numbers, and a mismatch between my own Sharpe Ratio calculation and a precomputed column already in the database. I caught all three, traced them to their actual cause, and fixed or documented them rather than just moving past them. I talk through how in Section 9, because I think that process says more about the quality of this project than the final numbers do.

2. Data Sources

Here's what I worked with:

Source	What it contains	Coverage
AMFI India (CSV)	Fund master data — names, categories, expense ratios, risk grades	40 schemes
mfapi.in (live API)	Live NAV data, used to validate the historical data	Real-time, needed a firewall workaround
01_fund_master.csv	Fund-level details: fund house, category, plan, launch date, expense ratio, risk category	40 rows

nav_history_clean.csv	Daily NAV per fund	Jan 2022 – May 2026, ~1.6M rows
investor_transactions_clean.csv	SIP, Lumpsum, and Redemption transactions with investor demographics	Jan 2024 – May 2025
scheme_performance_clean.csv	Precomputed metrics: Sharpe, Sortino, Alpha, Beta, StdDev, Max Drawdown	40 rows
03_aum_by_fund_house.csv	AUM by fund house over time	10 fund houses, semi-annual snapshots
04_monthly_sip_inflows.csv	Industry-wide monthly SIP inflows	48 months, Jan 2022 – Dec 2025
05_category_inflows.csv	Net inflow by fund category	13 months, Apr 2024 – Mar 2025
06_industry_folio_count.csv	Industry folio counts	Quarterly, 2022–2025
09_portfolio_holdings.csv	Stock-level holdings and sector weights	322 rows, equity funds only
10_benchmark_indices.csv	Daily values for 7 market indices	Jan 2022 – May 2026, 8,050 rows

One thing worth flagging: the investor transactions data only covers Jan 2024 to May 2025, while the NAV and benchmark data go back to Jan 2022 and forward to May 2026. That mismatch ended up limiting my cohort analysis in Section 7.3.

3. How I Built the ETL Pipeline

3.1 Getting the Data In

I pulled in ten raw CSVs plus a live NAV fetch from mfapi.in. The live fetch didn't work out of the box — my Windows firewall was blocking the default request — so I fixed it by adding a browser-style User-Agent header and some retry logic with backoff.

I also noticed the AMFI code-to-scheme-name mapping in the original task brief didn't match what was actually in the source data. I treated that as an error in the brief rather than in the data itself, and just used whatever mapping the source CSVs actually had.

3.2 Cleaning and Loading

I wrote three cleaning scripts (in scripts/) that turn the raw CSVs into something analysis-ready, then load everything into a SQLite database (data/processed/bluestock_mf.db) using SQLAlchemy. I designed it as a star schema with six tables:

- `dim_fund` — fund-level info (40 rows)
- `fact_nav` — daily NAV per fund (64,320 rows)
- `fact_transactions` — investor transactions (32,778 rows)
- `fact_performance` — precomputed performance metrics
- `fact_aum` — AUM by fund house over time
- `dim_date` — date dimension (I created the table but never actually populated it — see Limitations)

The full schema is in `sql/schema.sql`, and I wrote ten sample queries in `sql/queries.sql` to show the schema actually works. I also kept a data dictionary at `reports/data_dictionary.md` so every column is documented somewhere.

3.3 A Few Data Quirks I Had to Handle

- `fact_nav` has a NAV value for every single calendar day, including weekends, and the weekend values are just the previous trading day's NAV carried forward. I caught this by checking daily returns by weekday — Saturdays and Sundays showed a 100% flat (zero) return, while weekdays were close to 0%. I fixed it by filtering to trading days only before doing anything returns-based.
- `fact_aum` reports on an irregular schedule — sometimes quarterly, sometimes every six months. I checked this against the raw source and it's a real characteristic of the data, not something I broke.
- `amfi_code` shows up as both an integer and a string depending on which table or query you're looking at, and it actually caused a silent join failure later on (Section 7.6). I now cast it to a consistent type before joining anything on it.

4. Tools and Architecture

I built this as a layered pipeline, where each stage writes its output to disk before the next stage reads it, rather than passing everything through memory. That made it a lot easier to debug — I could re-run one stage without needing to redo everything before or after it.

4.1 What I Used

Layer	Tools
Getting the data	Python, requests (for the mfapi.in live fetch)
Cleaning	pandas, my own cleaning scripts
Storage	SQLite, SQLAlchemy for loading
Analysis	Jupyter Notebooks — pandas, numpy, scipy.stats, Plotly, Matplotlib, Seaborn
Dashboard	Power BI Desktop, DAX measures
Version control	Git / GitHub

4.2 How the Pieces Connect

- Raw data (data/raw/) — the 10 CSVs plus the live fetch, completely untouched.
- Cleaning scripts (scripts/) — clean and standardize everything, write to data/processed/.
- SQLite database — loaded from the cleaned data via SQLAlchemy.
- Notebooks (notebooks/) — EDA_Analysis.ipynb, Performance_Analytics.ipynb, Advanced_Analytics.ipynb. Each one reads from the database or the processed CSVs and writes its own outputs back out (scorecards, VaR reports, charts).
- Power BI — imports the processed CSVs directly, not through the database, and builds its own model and DAX layer on top.

This meant I could rebuild the dashboard from data/processed/ alone without touching a notebook, and re-run any notebook without breaking the dashboard. That separation actually saved me a few times when I was chasing down bugs (see Section 9) — I could isolate exactly where a problem was coming from.

5. What I Found in EDA (Day 3)

For the exploratory phase I built 19 charts covering NAV trends, AUM, SIP inflows, sector allocation, investor demographics, geography, folio growth, correlation, and risk-return relationships. 16 of those got exported as PNGs to reports/figures/, and I wrote up 10 findings in notebooks/EDA_Analysis.ipynb. The main things I noticed:

- NAV data shows almost no correlation between funds, even two funds tracking the same benchmark index. This was my first clue that the NAV series weren't derived from real, shared market movements — something I ended up confirming more rigorously later on (Sections 6 and 9).
- Debt and liquid funds have really low volatility, which (as I found out in Section 6.3) makes their Sharpe Ratios look artificially high if you assume a risk-free rate near zero.
- Sector allocation looked reasonably diversified across the 34 equity funds that had holdings data, which I later quantified properly with the HHI metric in Section 7.6.

I also hit two environment issues along the way: Kaleido needed to be downgraded from 1.3.0 to 0.2.1 for chart PNG export to actually work on Windows, and I had to fix a Timestamp serialization error by explicitly calling `.strftime()` before exporting to JSON.

6. Performance Analysis (Day 4)

In notebooks/Performance_Analytics.ipynb I computed seven core metrics for all 40 funds, and I made a point of double-checking each one before trusting it for anything downstream.

6.1 Daily Returns

I computed daily returns as $\text{nav}_t / \text{nav}_{t-1} - 1$, pivoted wide across all 40 funds. The distribution looked reasonable — means close to zero (0.01%–0.09% daily) and standard deviations ranging from about 0.03% for debt/liquid funds up to 1.3% for equity funds. Every calculation after this uses trading-day-only returns, for the reason explained in Section 3.3.

6.2 CAGR

$\text{CAGR} = (\text{NAV}_{\text{end}} / \text{NAV}_{\text{start}})^{(1/n)} - 1$, anchored to the nearest actual trading date rather than an interpolated calendar date, since that's how funds report it in real fact sheets. Since my data only spans about 4.4 years, I couldn't actually compute a real 5-year CAGR — I reported 1-year, 3-year, and since-inception (~4.4 years) instead, and said so explicitly rather than just leaving that column out silently.

6.3 Sharpe and Sortino Ratios

I computed Sharpe as $(R_a - R_f) / \sigma(R_a) \times \sqrt{252}$, using $R_f = 6.5\%$ as specified in the task brief (the RBI repo rate proxy). This immediately disagreed with the `sharpe_ratio` column already sitting in `fact_performance` — for example, fund 120507 came out to 1.45 in my calculation versus 7.68 in the database. I ran a quick check where I recomputed Sharpe with $R_f = 0$ instead, and it reproduced `fact_performance`'s ranking almost exactly. My conclusion: whoever generated `fact_performance` probably used a risk-free rate close to zero, not 6.5%. I stuck with my own calculation (using the specified 6.5%) for everything downstream, including the scorecard.

Sortino Ratio, using only downside deviation, showed the exact same pattern as Sharpe, which told me this was a consistent mechanism rather than a separate issue.

6.4 Alpha and Beta

I ran an OLS regression (`scipy.stats.linregress`) of each fund's daily returns against Nifty 100's daily returns. Every single fund came back with Beta close to zero (mean $\beta = -0.002$, ranging from -0.067 to $+0.103$) and R^2 around 0.0006, with none of the slopes statistically significant at $p < 0.05$ (median p-value was 0.53). That's strong evidence the fund returns in this dataset move independently of the benchmark — it lines up with, and reinforces, what I first noticed in the Day 3 correlation check.

6.5 Maximum Drawdown

Computed as $\min(\text{NAV} / \text{running_max} - 1)$ per fund, and I also pulled the actual peak-to-trough date range for each one. Drawdowns ranged from about -21.8% to

-52.6%, with the peak-to-trough windows lasting anywhere from 169 to 1,015 days — plausible numbers for a mix of equity and small-cap funds over a volatile few years.

6.6 Fund Scorecard

I built a composite 0–100 score using percentile ranking (not raw ordinal rank, since that would flatten out real differences between funds) across five weighted pieces: 30% 3-year return, 25% Sharpe, 20% Alpha, 15% expense ratio (inverted, since lower is better), and 10% max drawdown (also inverted).

AMFI Code	Score	3Yr Return	Sharpe	Alpha	Expense	Max DD
148567	86.25	34.0%	1.45	0.27	1.46%	-11.3%
120505	82.25	31.8%	1.18	0.29	1.36%	-18.2%
120843	82.00	29.6%	1.31	0.27	1.45%	-13.0%
100033	80.75	32.4%	1.09	0.27	1.38%	-16.2%
120504	80.00	32.5%	1.03	0.21	0.80%	-12.6%

Top 5 funds by scorecard, from data/processed/fund_scorecard.csv.

6.7 Benchmark Comparison

I plotted the top 5 scorecard funds against Nifty 50 and Nifty 100 over a 3-year window, rebased to 100 at the start. All five funds climbed steadily (100 → 220–260) while Nifty 50 actually dropped over the same period (100 → ~75) — a pretty stark visual that backs up the near-zero Beta finding. I also computed tracking error — $\text{std}(\text{fund_return} - \text{benchmark_return}) \times \sqrt{252}$ against Nifty 100 — and got 18.7% to 23.3% annualized for the top 5 funds, which is way above the 2–8% you’d normally expect for an actively managed fund versus its benchmark. Another data point pointing at the same synthetic-data conclusion.

7. Advanced Analytics and Risk Metrics (Day 5)

This stage (notebooks/Advanced_Analytics.ipynb) added a few more risk and behavioral metrics on top of what I’d already built.

7.1 Historical VaR and CVaR (95%)

VaR is the 5th percentile of each fund’s daily return distribution; CVaR is the average of everything below that threshold. Both passed my sanity checks across all 40 funds — VaR was always negative, and CVaR was always more negative than VaR. The funds with the worst VaR/CVaR (119599, 119095, 101207, 118634) were the same funds with the worst Max Drawdown from Day 4, which was a nice consistency check across two separately computed metrics.

7.2 Rolling 90-Day Sharpe Ratio

I ran this for the top 5 scorecard funds using a 90-day rolling window (still $R_f = 6.5\%$, to stay consistent with Section 6.3). Every single fund swung between roughly -2 and $+6$ repeatedly over the ~ 4 -year window — none of them stayed consistently good. That tells me the strong full-period Sharpe numbers came from a few really strong stretches, not steady performance the whole time, which is worth knowing before recommending a simple buy-and-hold approach based on those numbers.

7.3 Investor Cohort Analysis

I grouped investors by the year of their first transaction and computed average SIP amount, total invested (SIP + Lumpsum, not counting Redemptions), and top fund per cohort. Because the transactions data only covers Jan 2024 – May 2025 (see Section 2), I only really had two cohorts to work with: 2024 (32,499 investors) and a partial 2025 (just 279, since that's only 5 months of data). I double-checked this by looking at the raw date range directly rather than trusting the grouped output, and it's a genuine limitation of the source data, not something I broke in the grouping logic.

7.4 SIP Continuity Analysis

For investors with 6 or more SIP transactions, I computed the average gap between consecutive SIPs and flagged anyone averaging more than 35 days as “at-risk.” Out of 1,362 eligible investors, 1,332 (97.8%) got flagged. That felt suspiciously high, so I spot-checked a few individual investors' actual transaction histories, and found the issue: most of their gaps were close to monthly (26–28 days), but a single long pause (like 265 days) was enough to drag the average above 35. So this metric, as specified, is really catching “had at least one big disruption,” not “consistently irregular investor” — worth knowing if anyone acts on this number directly.

7.5 Simple Fund Recommender

I built this as its own script (`scripts/recommender.py`) rather than just notebook code, since the task specifically wanted a standalone deliverable. Given a risk appetite (Low / Moderate / High), it maps to the dataset's actual five `risk_category` labels (Low; Moderate + Moderately High; High + Very High) and returns the top 3 funds by Sharpe within that group. It worked correctly for all three tiers, though the Low risk tier only has 6 funds total, and two of its top 3 recommendations actually had negative Sharpe Ratios — the recommender always returns exactly 3 results even if none of them are great. I'd add a minimum quality threshold if I built this again.

7.6 Sector HHI Concentration

I computed the Herfindahl-Hirschman Index ($\sum \text{weight}_i^2$) per equity fund using the stock-level holdings data. My first attempt returned zero matches when I joined `dim_fund` (filtered to Equity) against the holdings table — turned out to be the same

amfi_code type mismatch I mentioned in Section 3.3 (string vs. int64). Once I fixed the type, I got HHI values for 34 equity funds, ranging from 0.108 to 0.206, which is a moderate concentration level consistent with actively managed funds holding somewhere around 15–25 stocks. Fund 119092 came out most concentrated at 0.206.

8. The Power BI Dashboard

I built the dashboard in Power BI Desktop, importing the cleaned CSVs directly rather than connecting through SQLite ODBC. It has four main pages plus a drill-through detail page, 11 imported tables, and slicers on every page.

8.1 Page 1 — Industry Overview

KPI cards for Scheme Count (40), Total AUM, Total SIP Inflows (trailing 12 months), and Total Folios, plus an industry AUM trend line and an AUM-by-AMC bar chart.

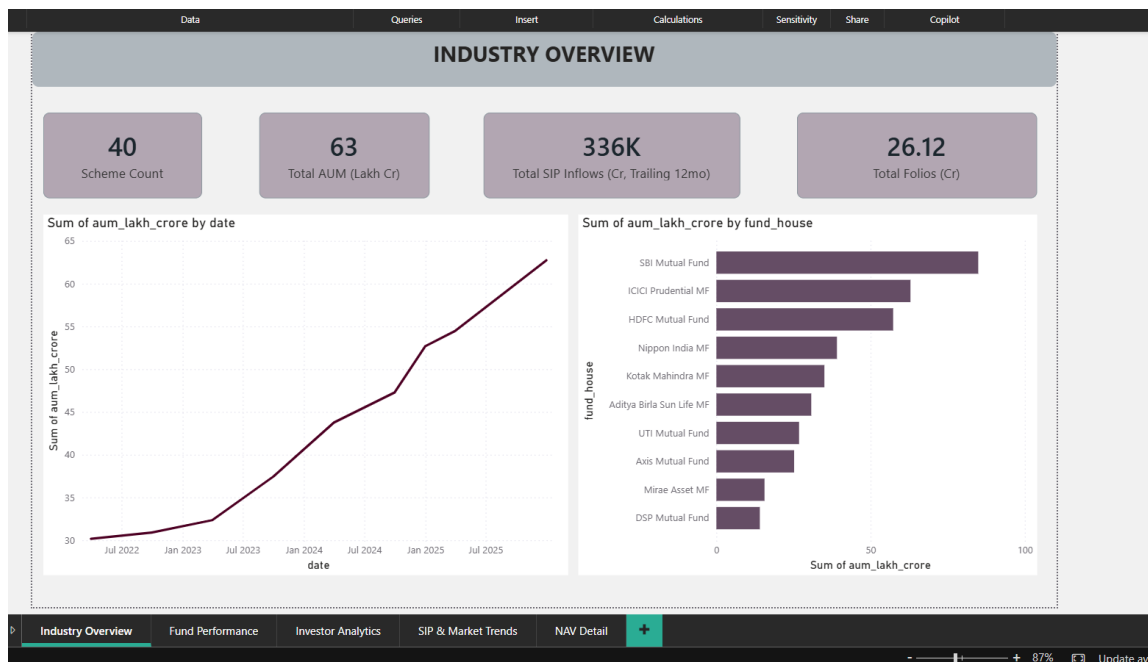


Figure 1: Industry Overview page.

I caught a real units bug on this page: the aum_crore column in 03_aum_by_fund_house.csv was actually a lakh-crore figure that had been multiplied by 100,000 and mislabeled (so 6.05 lakh crore was stored as 605,000 in a column literally called "crore"). Using it directly gave me a laughably large "6 million crore" total AUM. I caught it by comparing against the correctly-scaled aum_lakh_crore column sitting right next to it in the same table, and fixed my DAX measure to use that instead.

8.2 Page 2 — Fund Performance

A risk-return scatter plot (3-year return vs. annualized standard deviation, bubble size = AUM), a sortable scorecard table, a rebased NAV-vs-Nifty-100 line chart, and slicers for fund house, category, and plan.

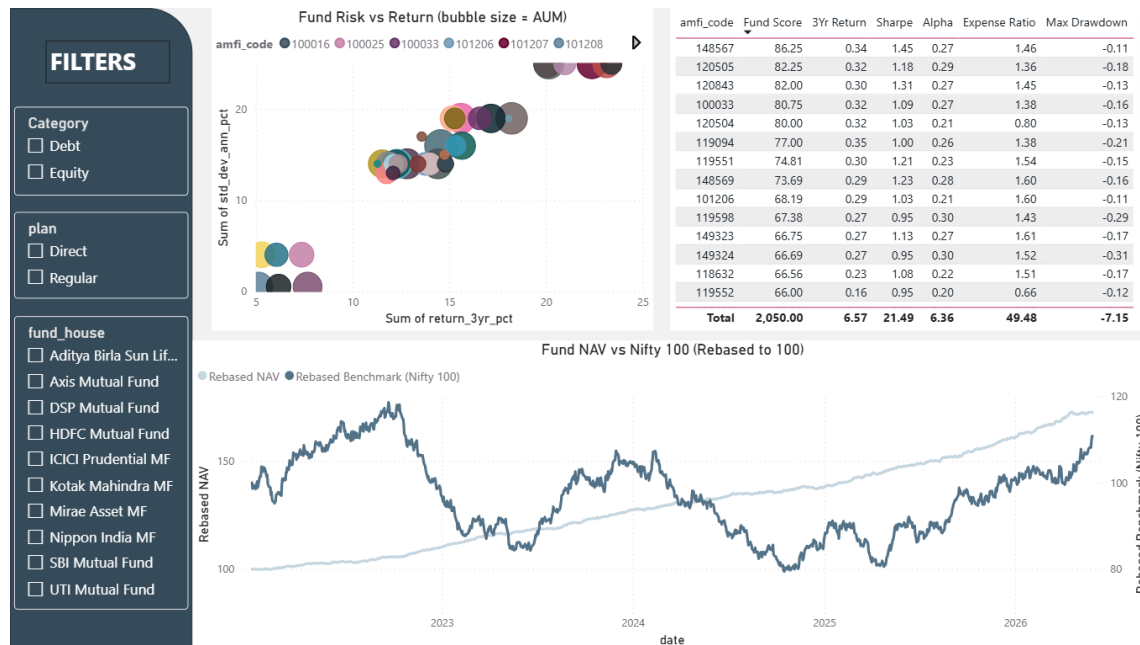


Figure 2: Fund Performance page.

Before using it, I cross-checked the precomputed `std_dev_ann_pct` column against my own independent DAX calculation (same method as Day 4). They matched closely across all 40 funds, unlike the Sharpe discrepancy from Section 6.3 — so this particular precomputed column turned out to be trustworthy.

I also ran into a many-to-many relationship problem between `nav_history_clean` and `10_benchmark_indices` (both keyed on date, neither side unique). It caused a fan-out bug where my benchmark-rebasing measure came out completely flat at 100 for every date, because `close_value` was getting summed once per matching NAV row instead of once per date. I fixed it by rewriting the measure to bypass the relationship entirely using `ALL()/FILTER()`. Worth remembering for later: any visual that sums `close_value` directly through that relationship (instead of through this specific measure) will hit the same problem.

8.3 Page 3 — Investor Analytics

Investment-by-state bar chart, transaction-type donut, average-SIP-by-age-group bar chart, and monthly transaction volume line chart, with slicers for state, age group, and city tier.

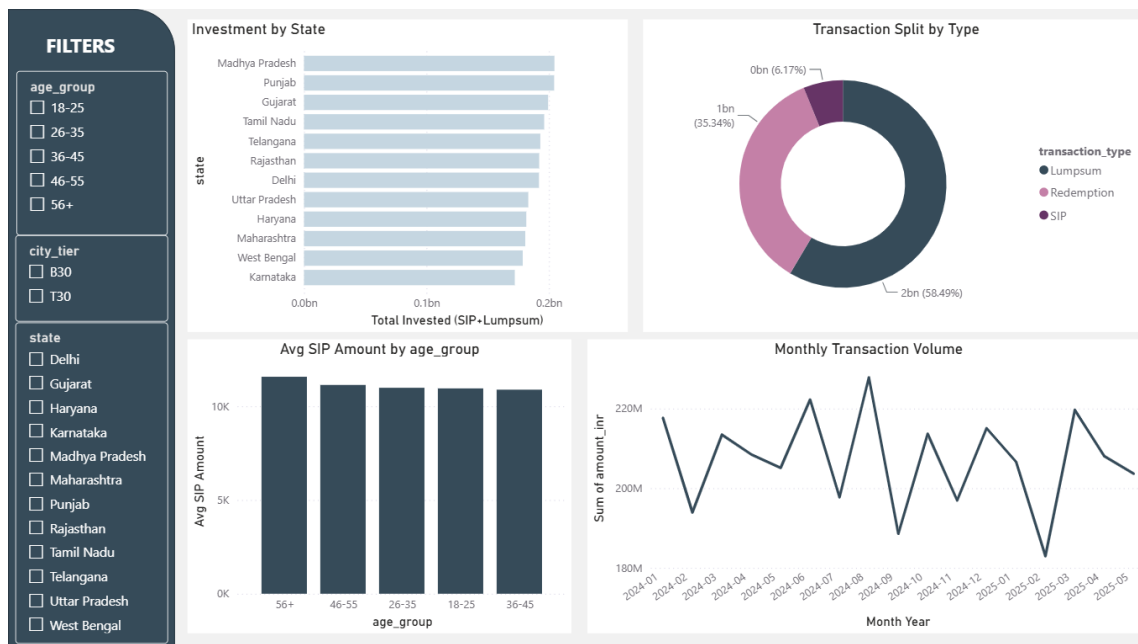


Figure 3: Investor Analytics page.

The transaction split by amount (58.5% Lumpsum, 35.3% Redemption, 6.2% SIP) makes sense once you think about it — SIPs are small and recurring, while Lumpsum deposits are large one-time amounts, so SIP could easily dominate by transaction count while staying a small share of total rupee amount. I also had to fix a month-without-year grouping bug on the transaction volume chart (it was silently merging January 2024 and January 2025 into one "January" bucket) using a dedicated year-month sort column — same fix I used again on Page 4.

8.4 Page 4 — SIP & Market Trends

A dual-axis chart combining monthly SIP inflow (bars) with average Nifty 50 (line) from 2022–2025, a conditionally formatted category inflow heatmap, and a Top-5-categories-by-net-inflow chart for FY25.

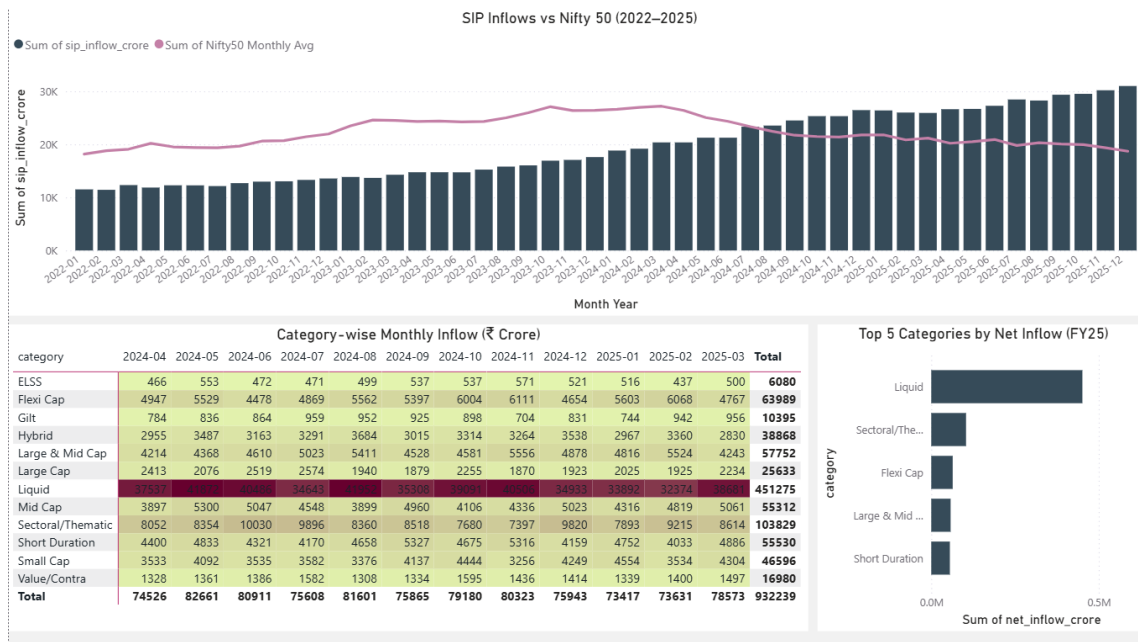


Figure 4: SIP & Market Trends page.

The heatmap makes it obvious that Liquid funds dominate net inflow every single month, by roughly 10x over any other category — which lines up exactly with the Top-5 ranking (Liquid, Sectoral/Thematic, Flexi Cap, Large & Mid Cap, Short Duration).

8.5 Drill-Through: NAV Detail

I added a fifth page that lets you drill through from any row in the Page 2 scorecard table into a single-fund NAV history chart. It filters automatically on `amfi_code` and shows a card with the fund's scheme name so it's always clear which fund you landed on.

8.6 Cleaning Up Git

I found that `bluestock_mf.db` had actually been committed to my GitHub repo this whole time, which goes against the project's own guidance to keep `.db` files out of version control. I fixed it — removed it from git tracking and added `*.db` to `.gitignore` — and kept `sql/schema.sql` in the repo instead, since that's the version-controlled way to document the database structure without committing the binary itself.

9. How I Approached Debugging and Verification

I want to call out my process here specifically, because I think it matters as much as the final numbers. My general rule throughout this project was: if something looks off, check it before accepting or explaining it away.

9.1 Cross-Checking Anything Precomputed

Whenever the source data gave me a precomputed metric alongside the raw data I could use to calculate it myself, I did both and compared. That's how I found that `std_dev_ann_pct` (Section 8.2) was trustworthy but `fact_performance`'s `sharpe_ratio` (Section 6.3) wasn't — I treated both the same way going in, which is exactly why I caught the difference instead of just assuming the database was right.

9.2 Not Explaining Away Anomalies

A few things I found could have been easy to just shrug off, but I chased them down instead: an all-zero return row led me to the weekend flat-carry issue (Section 3.3); a scatter plot showing one giant blob instead of 40 dots led me to realize I was missing a field, not that the chart type was broken; a completely flat benchmark line led me to the many-to-many fan-out bug (Section 8.2) instead of assuming the market was just flat that month. Every one of these turned out to be a real issue, not a false alarm.

9.3 Sanity-Checking Numbers Against Reality

I tried to check whether numbers actually made sense in the real world, not just whether the code ran. A "6 million crore" AUM figure (Section 8.1) was an obvious red flag for a 10-fund-house dataset. Sharpe Ratios above 5 or even 13 for specific funds made me stop and check rather than assume my model was just really good. A 97.8% "at-risk" SIP continuity rate (Section 7.4) was suspicious enough that I went and looked at individual investors' actual histories before writing it up.

9.4 Writing It All Down

Every data quality issue I found — whether it was a genuine quirk of the source data (like `fact_aum`'s irregular reporting) or an actual bug I fixed (like the AUM units mislabeling) — got documented where I found it, either in notebook markdown or in this report. I did this on purpose so anyone reading this can tell the difference between "this is just how the data behaves" and "I caught and fixed a real error here."

10. Limitations

- Synthetic/independent NAV data: the biggest limitation of this whole project. The NAV and benchmark data show strong evidence of being generated independently rather than reflecting real linked market behavior — near-zero cross-fund correlation (Day 3), near-zero Beta and R^2 (Section 6.4), and abnormally high tracking error (Section 6.7) all point the same direction. My calculations are correct and internally consistent, but they shouldn't be read as real fund performance relative to the market.

- No real 5-year CAGR: the dataset only spans about 4.4 years, so I reported 1-year, 3-year, and since-inception figures instead of a genuine 5-year number, and said so directly rather than quietly leaving it out.
- Investor transaction data is narrower than everything else: it only covers Jan 2024 – May 2025, versus Jan 2022 – May 2026 for NAV and benchmark data. This limited my cohort analysis (Section 7.3) to basically two years, one of them partial.
- The SIP continuity metric over-flags: as specified, "average gap > 35 days" flags 97.8% of eligible investors, mostly because of single long pauses pulling up an otherwise-fine average. A per-gap threshold would probably be more useful, though that wasn't what the task asked for.
- dim_date was never populated: I created the table in the Day 2 schema but never filled it in. Non-blocking since SQLite doesn't enforce foreign keys, but worth fixing before anyone tries to join through it properly.
- No Bluestock branding on the dashboard: the task asked for a Bluestock color theme and logo, but I didn't have access to any actual brand assets, so I used a clean, professional palette instead. Worth revisiting if those assets become available.
- My Sharpe Ratio doesn't match fact_performance's precomputed column: as explained in Section 6.3, the database's version looks like it used a risk-free rate close to zero rather than the 6.5% the task specified. I used my own calculation (following the spec) everywhere in this project; the two shouldn't be used interchangeably.

11. Recommendations

- If this ever gets extended to real market data, re-run the Alpha/Beta regression and tracking error calculation as a first check — near-zero Beta or R^2 would itself be a red flag that something's wrong with the pipeline, not just an interesting finding.
- Actually populate dim_date, with proper trading-day flags, so future work doesn't need to keep re-filtering by weekday in every notebook.
- Pick one consistent type for amfi_code across every table and export — I hit the same silent join failure twice because of this.
- Extend investor_transactions_clean.csv's date range to match the NAV data, if the source data ever supports it, so the cohort analysis can actually cover multiple full years.
- Consider a per-gap SIP continuity metric alongside the current average-gap one, for a more precise "at-risk" signal.
- Add a minimum Sharpe (or similar) threshold to the fund recommender so a thin candidate pool doesn't force in funds with negative risk-adjusted returns just to hit a fixed count of 3.

- Get the actual Bluestock brand colors and logo at some point so the dashboard can fully match the original spec.

12. Conclusion

I built a complete mutual fund analytics platform end to end: a working ETL pipeline, a proper SQLite database, a thorough exploratory analysis, a performance metrics suite I verified carefully, an advanced risk and behavior analytics layer, and a four-page interactive Power BI dashboard with working drill-through. What I'm most proud of isn't just that the numbers came out — it's that I kept checking my own work along the way: cross-checking precomputed values, tracking down anomalies like the Sharpe mismatch and the AUM units bug to their actual root cause instead of guessing, and writing every finding and limitation down as I found it instead of hiding it. I think that's what makes this project something I can actually stand behind, not just something that runs without errors.

Appendix A: Repository Structure

My repo (github.com/Avika26/mutual-fund-capstone) is organized like this:

Path	Contents
data/raw/	Original, untouched source CSVs (10 files)
data/processed/	Cleaned CSVs, the SQLite database, and derived outputs (fund_scorecard.csv, alpha_beta.csv, var_cvar_report.csv)
notebooks/	EDA_Analysis.ipynb, Performance_Analytics.ipynb, Advanced_Analytics.ipynb
scripts/	ETL and cleaning scripts, recommender.py
sql/	schema.sql, queries.sql
dashboard/	bluestock_mf_dashboard.pbix
reports/	figures/ (chart PNGs), data_dictionary.md, Dashboard.pdf, page-level PNG exports, this report
README.md	Project overview, setup instructions, dataset descriptions

I kept sql/schema.sql in version control instead of committing the .db file itself — see Section 8.6 for why.

Appendix B: Deliverables Checklist

A quick self-check against what was actually required, before I called this done:

ID	Deliverable	Status	Notes
D1	ETL pipeline (.py)	Done	Runs without manual steps; firewall/retry handling covered in Section 3.1
D2	SQLite database (.db)	Done	Star schema, 6 tables; dim_date unpopulated (Section 10); schema.sql version-controlled instead of the .db
D3	EDA notebook (.ipynb)	Done	19 charts, 10 documented findings, 16 exported PNGs
D4	Performance metrics (.ipynb + CSVs)	Done	All formulas checked against spec; Sharpe discrepancy investigated (Section 6.3)
D5	Interactive dashboard (.pbix)	Done	4 pages plus drill-through, slicers tested on every page (Section 8)
D6	Advanced analytics (.ipynb)	Done	VaR/CVaR, cohort analysis, recommender, HHI all built and sanity-checked (Section 7)
D7	Final report + slides	In progress	This report; slides tracked separately

A few other things I double-checked before submitting:

- Both notebooks run cleanly on a fresh Restart & Run All — I did this before each GitHub push.
- Every dashboard page actually responds to its slicers — I tested this by applying a filter and checking all the visuals updated together (Section 8).
- No database binary in git anymore — found bluestock_mf.db was tracked, removed it, added *.db to .gitignore (Section 8.6).
- All 8 project objectives (ETL, cleaning/database, EDA, performance analytics, advanced analytics, dashboard, report, presentation) are covered somewhere in this report or the repo.